DOCUMENTACIÓN TÉCNICA

BarberTrend AI

Sistema de reservas y marketing con IA para Doble S Barbería

Estado: Etapa 3 cerrada · avisos funcionando

Fecha: 23 de octubre de 2026

Stack: Java 21 · Spring Boot 4.1.1 · H2 / PostgreSQL

Pruebas: 30 de 30 en verde

Documento: alcance actual + hoja de ruta

Proyecto académico y de uso real. Los datos del negocio que figuran aquí fueron aportados y confirmados por el dueño; el resto está rotulado como dato de ejemplo.

Contenido

1 Resumen ejecutivo

2 El producto

3 Datos reales del negocio

4 Stack tecnológico

5 Arquitectura

6 Estado por etapas

7 Lo que ya está hecho

8 La garantía anti-superposición de turnos

9 Seguridad

10 API REST

11 Modelo de datos

12 Pruebas automatizadas

13 Cómo ejecutar y probar

14 Configuración (variables de entorno)

15 Hoja de ruta: lo que viene

16 Deuda técnica y límites conocidos

17 Datos del negocio pendientes

> **Cómo leer este documento.** Las secciones 1 a 6 dan el panorama. Las secciones 7 a 12 explican lo implementado y por qué. Las secciones 13 a 17 son prácticas: cómo correrlo, cómo configurarlo, qué falta y hacia dónde va.

1. Resumen ejecutivo

BarberTrend AI es una aplicación web para una barbería masculina que combina dos módulos: un **sistema de reservas online** y, más adelante, un **asistente de marketing con IA** para detectar tendencias y generar contenido para redes.

Hoy el primero de esos módulos ya está operativo de punta a punta:

- La web pública muestra los servicios, precios, horarios, galería y contacto con **datos reales** de la barbería.
- Un cliente puede **reservar un turno de verdad**. La disponibilidad la calcula el backend y el sistema garantiza que dos personas no puedan quedarse con el mismo horario.
- El barbero entra a un **panel privado**, ve los turnos y los puede **confirmar, completar o cancelar**.
- Cuando entra un turno nuevo, el barbero recibe un **aviso por email** en forma automática.
- Al confirmar, el panel le da un **enlace de WhatsApp** con el mensaje ya escrito para avisarle al cliente: se envía con un toque, sin costo y sin depender de la API de Meta.

El sistema es **100 % gratuito** en su operación actual: no usa servicios pagos, y el único requisito para recibir emails es una cuenta de Gmail con contraseña de aplicación (también gratuita).

> **Alcance de este documento.** Describe todo lo construido y verificado hasta la fecha y traza el camino de las etapas siguientes. No marca como terminado nada que no esté probado: el estado de cada punto está indicado con `Listo`, `Parcial` o `Pendiente`.

2. El producto

El proyecto se pensó como un producto útil para un negocio real (una barbería familiar) y, a la vez, como proyecto de portfolio reutilizable para otras barberías. La estrategia fue construir por etapas, sin incorporar tecnologías nuevas hasta que hicieran falta.

2.1 Sistema de reservas

- Web pública responsive con catálogo de servicios y precios.
- Selección de fecha y horario, con disponibilidad calculada en el servidor.
- Reserva de turnos con datos básicos del cliente.
- Garantía de que no se superpongan dos turnos.
- Consulta del estado de un turno mediante un código.
- Panel administrativo privado para gestionar la agenda.
- Avisos al barbero y al cliente.

2.2 Asistente de marketing (etapa posterior)

- Detección de tendencias del rubro a partir de fuentes verificables.
- Ideas y guiones breves para Reels e historias.
- Captions y hashtags; recomendaciones periódicas.
- Historial de sugerencias y registro de resultados.

> **Regla de oro del proyecto:** la IA debe generar contenido a partir de información disponible y verificable. No se inventan tendencias, estadísticas, enlaces ni métricas. Cada dato se distingue entre *verificado*, *estimado*, *sugerencia creativa* o *no obtenido*.

3. Datos reales del negocio

Ninguno de estos datos es inventado: fueron aportados o confirmados por el dueño y ya están cargados en la aplicación.

Nombre	Doble S Barbería
Instagram	@Barberia_doble_s
WhatsApp	+54 9 2284 239845
Teléfonos	2284 239845 · 2284 245945
Dirección	Bolívar 3008 *(guardada; se usa en los mensajes de confirmación)*
Logo	PNG con fondo transparente (generado a partir del JPEG original)
Galería	4 fotos reales de cortes, optimizadas

Horario de atención

Día	Horario
Lunes a sábado	09:00 – 21:00
Domingo	Cerrado

Catálogo de servicios (precios reales)

Servicio	Duración interna	Precio
Corte clásico	30 min	$ 12.000
Barba completa	30 min	$ 12.000
Corte + barba	60 min	$ 15.000
Corte infantil	30 min	$ 9.000

> **Las duraciones no se muestran al cliente.** Un corte no dura siempre lo mismo. Pero el dato *sí* existe en el modelo, porque el sistema lo necesita para saber cuántos bloques de 30 minutos ocupa el turno y así calcular la disponibilidad y evitar superposiciones.

Lo único que sigue siendo de ejemplo

Los textos de portada y el eslogan. Son redaccionables y se reemplazan sin tocar código.

4. Stack tecnológico

Componente	Elección	Motivo
Lenguaje	Java 21 (LTS)	Versión estable y de soporte largo; base de Spring Boot.
Framework backend	Spring Boot 4.1.1	Última estable al iniciar. Trae servidor web, seguridad, JPA y validación integrados.
Build	Maven (wrapper `mvnw`)	No requiere instalar Maven: el wrapper lo descarga.
Base de datos · desarrollo/pruebas	H2	Cero instalación. En pruebas es en memoria (aislada); en desarrollo, en archivo.

Base de datos · producción	PostgreSQL 18	Datos concurrentes, persistentes y respaldables.
Migraciones	Flyway	Un único juego de SQL para ambos motores. El esquema nunca lo cambia Hibernate.
Frontend	HTML, CSS y JavaScript	Sin frameworks ni dependencias: liviano, rápido en celulares y fácil de mantener.
Seguridad	Spring Security (sesión + BCrypt)	Suficiente y simple para un panel único.
Email	SMTP (Gmail en desarrollo)	Gratuito con contraseña de aplicación.

> **Decisiones que se evitaron a propósito:** React, microservicios, Redis, Docker y cualquier servicio pago. No aportan valor real en esta escala y agregan complejidad y costo.

5. Arquitectura

5.1 Capas

El backend está separado en responsabilidades claras. Cada capa sólo conoce a la de abajo:

Capa	Paquete	Responsabilidad
Controladores	`web/`	Reciben la petición HTTP y devuelven JSON o páginas. No contienen reglas de negocio.
Servicios	`service/`	La lógica: disponibilidad, reserva, cambios de estado. Es donde vive la verdad.
Repositorios	`repository/`	Acceso a la base de datos con Spring Data JPA.
Entidades	`domain/`	El modelo persistente (turnos, clientes, servicios, etc.).
DTO	`dto/`	Los datos que viajan por la API. No exponen las entidades directamente.
Configuración	`config/`	Seguridad, propiedades, zona horaria.
Avisos	`notification/`	Email al barbero y armado del enlace de WhatsApp.
Errores	`common/`	Traduce excepciones a respuestas uniformes.

5.2 Estructura de carpetas

```
BarberTrend/
├── docs/                       documentación
├── .env.example                plantilla de variables (sin secretos)
└── backend/
    └── src/
        ├── main/
        │   ├── java/com/barbertrend/
        │   │   ├── common/       manejo de errores
        │   │   ├── config/       seguridad y propiedades
        │   │   ├── domain/       entidades
        │   │   ├── dto/          records de la API
        │   │   ├── notification/ email y WhatsApp
        │   │   ├── repository/   acceso a datos
        │   │   ├── service/      lógica de negocio
        │   │   └── web/          controladores REST
        │   └── resources/
        │       ├── application*.yml     configuración por perfil
        │       ├── db/migration/        migraciones Flyway
        │       └── static/              web pública y panel
        └── test/                        pruebas automatizadas
```

5.3 Perfiles y base de datos

Perfil	Base de datos	Uso
`test`	H2 en memoria	Pruebas: arranca y se tira en cada corrida.
`dev`	H2 en archivo	Desarrollo local sin instalar nada.
`prod`	PostgreSQL 18	Producción.

El esquema se crea **sólo** con migraciones de Flyway. La propiedad `ddl-auto` está en `none`: Hibernate nunca modifica el esquema por su cuenta.

5.4 El frontend vive dentro de Spring Boot

El sitio y el panel son archivos estáticos servidos por la misma aplicación. Esto simplifica el despliegue (un solo servicio) y evita problemas de CORS. Cuando haga falta, se puede separar sin reescribir nada, porque el frontend sólo habla con la API por HTTP.

6. Estado por etapas

Etapa	Contenido	Estado
1	Análisis, arquitectura y modelo de datos	Listo
2	Web pública responsive	Listo
3	Backend, reservas, panel y seguridad	Listo
4	Integración con WhatsApp	Parcial — envío manual hecho; falta la API oficial
5	Asistente de marketing con IA	Pendiente
6	Automatización, métricas y despliegue	Pendiente

Qué funciona hoy, en una tabla

Parte	Estado

Diseño responsive (móvil, tablet, computadora)	Listo
Servicios, galería, horarios y contacto	Listo, con datos reales
Formulario de reserva	Listo: crea reservas reales
Backend de reservas	Listo, con garantía anti-superposición
Login y acceso al panel privado	Listo y probado
Panel: listar, confirmar, completar y cancelar turnos	Listo
Aviso por email al barbero	Listo (dry-run por defecto)
WhatsApp al cliente	Manual con enlace `wa.me`
Asistente de marketing con IA	Pendiente

7. Lo que ya está hecho

7.1 Etapa 1 — Análisis y arquitectura

- Elección y verificación de versiones compatibles.
- Modelo de datos completo, pensado para el negocio real.
- Definición de las reglas de agenda y de los estados de un turno.
- Documentación de decisiones, con motivos y fuentes.

7.2 Etapa 2 — Web pública

- Sitio responsive con portada, servicios, galería, horarios y contacto.
- Identidad visual definida (paleta oscura con dorado).
- Logo con fondo transparente, generado desde el JPEG original con una técnica de relleno por inundación para no perforar las partes blancas internas.
- Cuatro fotos reales de cortes, optimizadas (hasta un 89 % menos de peso).
- Todos los textos, precios y enlaces centralizados en `site-config.js`: cambiar la barbería no requiere tocar HTML ni CSS.

7.3 Etapa 3 — Reservas, panel y seguridad

Backend de reservas

- Cálculo de disponibilidad por servicio y fecha, aplicando horario de atención, anticipación mínima, horizonte máximo y turnos ya ocupados.
- Creación de turnos con validación de datos.
- Consulta pública del estado por código de referencia.
- Garantía de no superposición impuesta por la base de datos (ver sección 8).
- Errores uniformes con códigos claros (`SLOT_NOT_AVAILABLE`, `OUTSIDE_SCHEDULE`, `TOO_SOON`, `VALIDATION_ERROR`, etc.).

Sitio público conectado

- El formulario consulta la disponibilidad real y crea el turno.
- Estados de carga, éxito y error bien visibles.
- Si otro cliente tomó el horario, se muestra un mensaje claro y se recarga la grilla.
- Al reservar, se muestra el código para consultar el turno.

Panel del barbero

- Acceso privado con login y contraseña (hash BCrypt).
- Listado de turnos con filtro por estado (pendientes, confirmados, cancelados) y por rango de fechas.
- Acciones: confirmar, completar, cancelar (con motivo obligatorio) y abrir WhatsApp.
- Los teléfonos se muestran enmascarados en el listado y completos sólo en el detalle, que es donde hacen falta.
- Cierre de sesión por POST (no por un enlace que podría dispararse sin querer).

Avisos

- **Email automático al barbero** al entrar un turno nuevo.
- **WhatsApp manual al cliente** con el mensaje ya armado, al confirmar.
- Modo *dry-run*: sin credenciales, el email se escribe en el log y no se envía nada.

> **Un bug real que se encontró y se corrigió.** Las pruebas de concurrencia fallaban: ante reservas simultáneas, varios turnos quedaban con el mismo bloque. La causa era que JPA usaba `merge` en lugar de `insert`. Se corrigió implementando `Persistable` en la entidad del bloque. Hoy las pruebas de concurrencia pasan. El detalle está en la sección 8.3.

8. La garantía anti-superposición de turnos

Esta es la parte más importante del sistema y la que justifica varias decisiones de diseño. Como es un negocio real, dos personas reservando el mismo horario a la vez es exactamente lo que va a pasar.

8.1 El problema del enfoque obvio

El código "pregunto si está libre y, si está, lo grabo" tiene una **condición de carrera**: entre la consulta y el guardado entra la otra petición, que también ve el horario libre, y las dos graban.

8.2 La solución: bloques ocupados

La jornada se divide en bloques de 30 minutos. Una reserva no ocupa un instante: **ocupa bloques**. Un corte de 60 minutos a las 10:00 inserta dos filas: `(barbero, 10:00)` y `(barbero, 10:30)`.

La tabla `appointment_slot` tiene clave primaria compuesta `(barber_id, slot_start)`. Cuando alguien intenta tomar un bloque ya ocupado, el `INSERT` viola esa clave y la base de datos rechaza la operación. La aplicación traduce ese rechazo a `409 Conflict`.

Por qué no alcanza con un `UNIQUE(barber_id, starts_at)`: dos turnos pueden empezar en instantes distintos y pisarse igual.

```
10:00 → 11:00   y   10:30 → 11:30
Los dos starts_at son distintos, pero las dos personas quedan en la misma silla.
```

Ocupar bloques hace que el conflicto se vea *siempre*, sin importar cómo se combinen las duraciones.

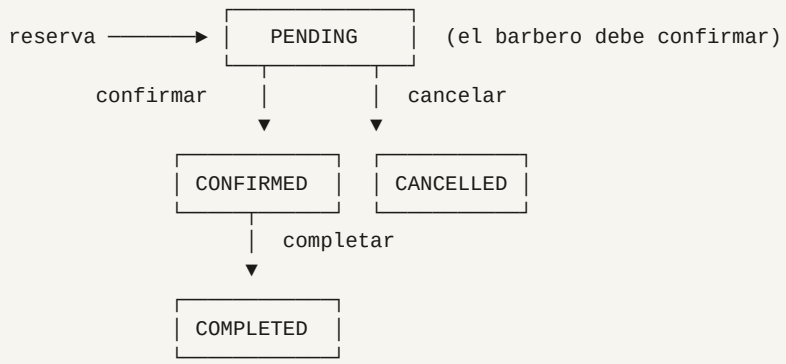
8.3 El bug que apareció en las pruebas

La primera versión usaba `saveAndFlush(...)`. Las pruebas de concurrencia fallaban: en lugar de una sola reserva exitosa, quedaban muchas. La causa:

- Con una **clave compuesta asignada a mano**, Spring Data no ve la entidad como "nueva" (la clave no es nula), así que usa `merge`.
- El `merge` primero consulta; si otro hilo ya insertó el bloque, **actualiza la fila existente** en lugar de fallar.
- Resultado: el que perdía la carrera se quedaba con el bloque del ganador.

La corrección fue implementar `Persistable` y marcar la entidad como nueva. Así Spring Data usa `persist` y el `INSERT` choca con la clave primaria. Las pruebas de concurrencia (20 peticiones simultáneas al mismo horario) pasan.

8.4 Estados de un turno



Estado	Significado	¿Ocupa el horario?
`PENDING`	Reserva recibida, esperando confirmación	Sí
`CONFIRMED`	Confirmada por el barbero	Sí
`COMPLETED`	El servicio se prestó	Ya no
`CANCELLED`	Cancelada, con motivo	No (libera el horario)

> Un turno `PENDING` **sí bloquea el horario**. Si dos personas piden las 10:00, la segunda recibe `409` aunque la primera todavía no se haya confirmado. Sin esa protección, ambos turnos entrarían a la lista de pendientes.

9. Seguridad

Medida	Implementación
Credenciales fuera del código	Variables de entorno en `.env` (ignorado por Git). `.env.example` sólo tiene valores de ejemplo.
Contraseña del panel	Hash BCrypt. Nunca texto plano (salvo modo desarrollo explícito, que avisa por log).
Rutas privadas	`/admin/**` exige sesión; `/api/admin/**` responde `401` sin sesión.
Sesión	Cookie `HttpOnly`; se cierra con `POST /admin/logout`.
CSRF	Activado. El token viaja en la cookie `XSRF-TOKEN` y en el campo `_csrf`. La API pública queda exceptuada.
Datos personales	Sólo se guarda lo indispensable (nombre y teléfono). Los teléfonos se enmascaran en los logs y en los listados.
Validación	En el servidor, no sólo en JavaScript.
Errores	Respuestas uniformes; nunca se exponen detalles internos ni trazas al navegador.

> **Nota sobre la contraseña.** Si en algún momento se sube un `.env` por error, no alcanza con borrarlo: hay que *rotar* las credenciales, porque siguen en el historial de Git.

10. API REST

Todas las rutas responden y consumen JSON, salvo las páginas HTML del panel. Los errores tienen siempre la misma estructura:

```
{
  "code": "SLOT_NOT_AVAILABLE",
  "message": "Ese horario acaba de ser reservado. Elegí otro.",
  "field": null,
  "timestamp": "2026-10-23T22:30:00"
}
```

10.1 Endpoints públicos

Método	Ruta	Qué hace
GET	`/api/public/services`	Servicios activos, ordenados.
GET	`/api/public/availability`	Horarios libres para un servicio y una fecha.
POST	`/api/public/appointments`	Crea una reserva (nace `PENDING`).
GET	`/api/public/appointments/{reference}`	Consulta el estado de un turno por su código, sin sesión.

Ejemplo: crear una reserva

```
POST /api/public/appointments
{
  "serviceId": 1,
  "startsAt": "2026-10-24T10:00:00",
  "customerName": "Juan Pérez",
  "phone": "5491122334455",
  "notes": "corto los lados"
}

201 Created
{
  "reference": "BT-7K2M9Q",
  "status": "PENDING",
  "serviceName": "Corte clásico",
  "startsAt": "2026-10-24T10:00:00",
  "endsAt": "2026-10-24T10:30:00",
  "message": "Recibimos tu pedido. Te confirmamos el turno a la brevedad."
}
```

10.2 Endpoints del panel

Requieren sesión. Sin sesión, la API devuelve `401`.

Método	Ruta	Qué hace
GET	`/api/admin/appointments?from=&to=&status=`	Listado (por defecto: hoy + 7 días), filtrable por estado.
GET	`/api/admin/appointments/{id}`	Detalle con teléfono completo y enlace de WhatsApp.
PATCH	`/api/admin/appointments/{id}/confirm`	Pasa de `PENDING` a `CONFIRMED`.
PATCH	`/api/admin/appointments/{id}/complete`	Pasa a `COMPLETED` y libera el horario.

POST	`/api/admin/appointments/{id}/cancel`	Cancela con motivo y libera el horario.
GET / POST / PUT	`/api/admin/services`	Gestión del catálogo (incluye inactivos).
PATCH	`/api/admin/services/{id}/active`	Activar o desactivar un servicio.
GET / PUT	`/api/admin/business-hours`	Horario semanal de atención.

Detalle de un turno (extracto)

```
{
  "id": 42,
  "reference": "BT-7K2M9Q",
  "status": "CONFIRMED",
  "customerPhone": "2284239845",
  "clientWhatsAppUrl": "https://wa.me/5492284239845?text=Hola%20Juan..."
}
```

En el listado el teléfono va enmascarado; en el detalle va completo, porque es la pantalla donde el barbero lo necesita para avisarle al cliente.

11. Modelo de datos

Tablas principales:

Tabla	Para qué
`service`	Catálogo: nombre, descripción, precio, duración real, activo, orden.
`barber`	Barberos. Hoy hay uno, pero la tabla existe para no migrar datos después.
`business_hours`	Horario semanal: 7 filas, una por día. Los cerrados conservan su fila.
`customer`	Clientes: nombre y teléfono normalizado a dígitos. Sin datos de más.
`appointment`	Turnos: referencia pública, cliente, barbero, servicio, inicio, fin, estado, notas, motivo de cancelación.
`appointment_slot`	Bloques ocupados. Es la garantía de no superposición.
`time_off`	Bloqueos de agenda (vacaciones, feriados, almuerzos).

El código de referencia

Cada turno recibe un código corto y aleatorio (`BT-7K2M9Q`). Sirve para que el cliente consulte su turno sin iniciar sesión y sin ver los datos de otros. **No** es secuencial, porque eso permitiría adivinarlos.

Tablas previstas (aún no creadas)

Tabla	Etapa	Para qué
`marketing_source`	5	Fuentes de tendencias consultadas y su estado.
`marketing_idea`	5	Ideas generadas, con su origen verificable.
`marketing_delivery`	5	Envíos de recomendaciones y su resultado.
`post_result`	5+	Resultados de publicaciones, para relacionarlos con reservas.

12. Pruebas automatizadas

La suite se ejecuta con `./mvnw test`. Estado actual: **40 de 40 en verde**.

Clase de prueba	Cantidad	Qué cubre
`ScheduleRulesTest` (Límites y Redondeo)	9	Reglas de la grilla: horarios, duración y redondeo a bloques.
`ConcurrentBookingTest`	2	20 reservas simultáneas al mismo horario: gana exactamente una.
`BookingFlowApiTest`	3	Circuito completo: disponibilidad, alta, rechazo del duplicado, confirmación y WhatsApp.
`AdminBlockApiTest`	9	Cierres completos y parciales, superposición, turnos existentes, reserva directa bloqueada y permisos.
`LoginSecurityTest`	8	Acceso al panel, CSRF, redirecciones y cierre de sesión.
`WhatsAppLinkBuilderTest`	6	Normalización del teléfono argentino al formato internacional.
`EmailAppointmentNotifierTest`	3	Modo dry-run y contenido del email de aviso.

> **Lo que se probó manualmente además de la suite:** se levantó la aplicación, se reservó un turno desde la API pública, se verificó el rechazo del horario duplicado (`409`), se inició sesión en el panel, se confirmó y se canceló un turno, y se comprobó que el enlace de WhatsApp se genera con el número normalizado y los acentos correctos. También se abrió el panel en Chrome headless y se verificó que renderiza la sección **Cierres y bloqueos** y que carga la lista de cierres desde la API.

13. Cómo ejecutar y probar

Requisitos

- JDK 21 (por ejemplo, Adoptium Temurin).
- Maven 3.9 o superior; o usar el wrapper `mvnw`, que lo descarga solo.

Pasos

```
# 1) Copiar la plantilla de variables dentro de backend
cp .env.example backend/.env

# 2) Arrancar
cd backend
./mvnw spring-boot:run        # en Windows: mvnw.cmd spring-boot:run
```

Dirección	Qué hay
`http://localhost:8080`	Sitio público
`http://localhost:8080/admin/login`	Ingreso al panel del barbero
`http://localhost:8080/admin`	Panel (pide sesión)

Con el perfil `dev` no hace falta instalar una base de datos: usa H2 en archivo.

Ejecutar las pruebas

```
cd backend
./mvnw test
```

Probar el aviso por email sin enviar nada

Con `MAIL_ENABLED=false` (el valor por defecto), al reservar aparece en el log una línea como esta:

```
[MAIL dry-run] Turno BT-KSXMMH para 22******45 el 24/10/2026 10:00 hs
— servicio Corte clásico. Aviso a — (activá MAIL_ENABLED=true para enviarlo).
```

El teléfono aparece enmascarado: los datos personales no se escriben completos en el log.

14. Configuración (variables de entorno)

Todo lo configurable vive en `backend/.env`, que está ignorado por Git. La plantilla `.env.example` tiene los mismos nombres con valores de ejemplo.

Panel del barbero

Variable	Para qué
`ADMIN_USERNAME`	Usuario del panel.
`ADMIN_PASSWORD_HASH`	Contraseña como hash BCrypt (recomendado).
`ADMIN_PASSWORD`	Alternativa en texto plano, sólo desarrollo (avisa por log).

Agenda

Variable	Valor por defecto
`APP_TIMEZONE`	America/Argentina/Buenos_Aires
`APP_SLOT_MINUTES`	30
`APP_MIN_LEAD_MINUTES`	120 (anticipación mínima)
`APP_MAX_ADVANCE_DAYS`	60 (horizonte máximo)
`APP_PUBLIC_BASE_URL`	http://localhost:8080

Avisos por email

Variable	Para qué
`MAIL_ENABLED`	`false` = dry-run (no envía). `true` = envía de verdad.
`MAIL_HOST` / `MAIL_PORT`	Servidor SMTP (Gmail: `smtp.gmail.com` / `587`).
`MAIL_AUTH` / `MAIL_STARTTLS`	Autenticación y cifrado (Gmail: `true` / `true`).
`MAIL_USERNAME`	Cuenta desde la que se envía.
`MAIL_PASSWORD`	Contraseña de aplicación (16 caracteres), no la normal.
`MAIL_FROM`	Remitente. Si queda vacío, se usa `MAIL_USERNAME`.
`BARBER_EMAIL`	Casilla del barbero que recibe el aviso.
`BARBER_WHATSAPP`	WhatsApp del barbero (guardado para la etapa automática).

Activar el envío real de emails (gratis con Gmail).

1. Activá la verificación en dos pasos de tu cuenta de Google.
2. Creá una *contraseña de aplicación* (Cuenta de Google → Seguridad).
3. Completá en `backend/.env`: `MAIL_ENABLED=true`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM` y `BARBER_EMAIL`.
4. Reiniciá la aplicación.

Ojo con los acentos en el `.env`. El archivo se lee como ISO-8859-1, así que los acentos se rompen. El nombre comercial y la dirección se configuran en `application.yml` (que sí es UTF-8). Si hiciera falta ponerlos en el `.env`, hay que escaparlos (por ejemplo, `\u00ED` para la í).

15. Hoja de ruta: lo que viene

El orden previsto prioriza tener algo útil y confiable antes que sumar tecnología. Las etapas marcadas como hechas ya están verificadas; el resto es planificación.

Etapa 4 — Integración con WhatsApp Parcial

Hecho: envío manual con enlace `wa.me` y mensaje prearmado.

Falta:

- Evaluar la API oficial de WhatsApp Business (requiere cuenta de Meta, número dedicado y plantillas aprobadas; puede tener costo).
- Implementar un servicio de notificaciones desacoplado del proveedor.
- Reintentos y control de duplicados.
- Recordatorios automáticos, si aportan valor.
- Responder/cancelar turnos por WhatsApp.

Etapa 5 — Asistente de marketing con IA Pendiente

- Definir el perfil de la barbería y los estilos de corte.
- Diseñar una interfaz de IA intercambiable entre proveedores.
- Generar ideas, guiones para Reels, captions y llamadas a reservar.
- Guardar las ideas y evitar sugerencias repetidas.
- Distinguir explícitamente datos verificados, estimaciones y sugerencias creativas.

Etapa 6 — Fuentes de tendencias Pendiente

> **Hallazgo importante.** La API oficial de Instagram *no* ofrece búsqueda de hashtags ni de tendencias: sólo da insights de las publicaciones propias, publicación de contenido, moderación de comentarios, menciones y mensajería. Por lo tanto, el asistente no puede sacar tendencias de Instagram por API oficial. Habrá que usar fuentes reales y verificables (por ejemplo, YouTube Data API, que permite búsqueda con cupos, o feeds RSS del rubro), documentando fuente por fuente permisos, límites y costos *antes* de escribir código.

Etapa 7 — Automatización Pendiente

- Búsquedas programadas en la zona horaria de Argentina.
- Generación automática de recomendaciones y resumen por WhatsApp.
- Evitar ejecuciones simultáneas; reintentos controlados.
- Registros de actividad y opción de pausar.

Etapa 8 — Seguimiento y métricas Pendiente

- Registrar qué ideas se usaron y cuándo se publicaron.
- Cargar métricas manualmente y relacionarlas con turnos.
- Comparar formatos y orientar las próximas recomendaciones.

No se afirmará que una publicación generó una reserva si no hay una forma razonable de relacionarlas.

Etapa 9 — Despliegue Pendiente

- Comparar proveedores de alojamiento y estimar costos mensuales.
- Configurar variables de entorno y base de datos persistentes.

- Publicar la web con HTTPS.
- Probar reservas reales controladas y verificar persistencia.
- Documentar mantenimiento y recuperación ante errores.

> **Mejoras cercanas al sistema de reservas** (antes o junto con la Etapa 4): extender un turno desde el panel cuando un corte se pasa de hora, y avisar de los horarios que quedaron apretados al día siguiente.

16. Deuda técnica y límites conocidos

Deuda asumida de forma consciente, con su motivo y cuándo se resolvería.

Deuda / límite	Motivo	Se resuelve
Un solo usuario administrador (en memoria)	Una barbería tiene un dueño	Si aparece un segundo usuario
Sin selector de barbero en el frontend	Hoy hay un solo barbero	Cuando haya más de uno
Sin *rate limiting* en el formulario público ni en el login	Uso interno y volumen bajo	Antes de exponer a internet
Sin paginación	Volumen esperado bajo	Cuando el listado crezca
Sin caché	Volumen esperado bajo	Si se detecta latencia
Aviso por email síncrono (después del commit)	Simplicidad; el volumen es bajo	Con una cola/outbox, si hace falta garantizar el envío
El número del barbero no se usa aún para enviar	El aviso automático hoy es el email	En la etapa de WhatsApp automático
Servicios y horarios se editan sólo desde la API/base, no desde el panel	El panel de gestión llega después	Próximos pasos del panel

La prioridad es entregar un producto útil y confiable, evitando complejidad innecesaria.

17. Datos del negocio pendientes

El proyecto no inventa datos reales de la barbería. Cuando falta algo, el sitio lo oculta o muestra un valor marcado como ejemplo. Estado actual:

Dato	Estado
Nombre, logo, colores, Instagram	Real
WhatsApp y teléfonos	Real
Dirección (Bolívar 3008)	Real, guardada
Horario de atención	Real
Catálogo de servicios y precios	Real
Fotos de la galería	Reales (4)
Nombre del barbero	Pendiente
Email de contacto público	Pendiente
Política de cancelación	Pendiente

Textos de portada y eslogan	**De ejemplo**
Dominio y alojamiento	**Pendiente** (Etapa 9)
Credenciales y plantilla de Meta (WhatsApp)	**Pendiente** (Etapa 4)
Proveedor de IA y credenciales	**Pendiente** (Etapa 5)

> **Compromiso.** Mientras un dato esté pendiente, no se escriben precios, direcciones, horarios ni métricas inventadas como si fueran reales. Cuando el dueño aporte el dato, se carga en el archivo o la migración que corresponda y se indica exactamente qué se tocó.

Cierre

BarberTrend AI tiene hoy un sistema de reservas completo y en uso: el cliente reserva, el barbero administra su agenda desde el teléfono y recibe un aviso automático por cada turno nuevo. Todo esto funciona sin costo operativo.

Lo que sigue apunta a reducir trabajo manual (automatizar el WhatsApp y los recordatorios), a llevar el sistema a internet de forma segura y, en paralelo o después, a sumar el asistente de marketing con IA sobre fuentes de datos verificables.

Documento generado el 23 de octubre de 2026. Fuente editable: `docs/documentacion.html`. Para regenerar el PDF: `google-chrome --headless --print-to-pdf=Documentacion-BarberTrend.pdf docs/documentacion.html`.